enabling asynchronous form submissions (AJAX), which sends data to the server in the background without reloading the page, resulting in a smoother, more interactive user experience.

From a security perspective, overriding the default behavior lets developers add extra checks, such as CAPTCHA (Completely Automated Public Turing test to tell Computers and Humans Apart) validation or additional input verification before submission. However, client-side validation can be bypassed, so it is essential to implement server-side validation and security measures to ensure data integrity and protect against malicious submissions. Overriding the default behavior does not eliminate the need for these server-side protections.

For this topic, are you able to answer these follow-up/probing questions?

- "What are the main differences between the GET and POST methods in HTML forms?"
- "What happens if the `action` attribute is omitted from a form element?"
- "How can you override the default form submission behavior using JavaScript, and why might you want to do so?"
- "What best practices should you follow to ensure that sensitive data is protected when submitting forms?"
- "How the `method` and `action` attributes interact with other form attributes like `enctype` and `target`?"

React 19 introduced the `useActionState` hook (https://react.dev/reference/react/useActionState), which streamlines form submission by managing the full lifecycle of a form action. This hook handles asynchronous submissions, loading states, error management, and feedback display, all while preserving a declarative programming style. It does not use onSubmit handler,

Declarative programming describes how the UI should appear based on state, instead of specifying step-by-step updates. With React's `useActionState`, developers define UI responses to states (like loading indicators or errors), and React automatically handles DOM updates. This state-driven approach simplifies code readability, maintenance, and reasoning.